מחוון ודוגמאות ניכוי – פעילות מעקב חיות בר

מסמך לבודק – Full Stack Wildlife Tracking System

כללי בדיקה

- **לא מריצים את הקוד.** בודקים את הלוגיקה והמבנה מתוך קריאה.
- **שגיאות תחביר קלות:** בדרך כלל 0.5–1 נקודה בסך הכול, כל עוד הכוונה ברורה.
- **לוגיקה חשובה יותר מתחביר.** מורידים יותר רק כשהגישה שגויה בבירור או לא עונה על הנדרש.
- **פיצ'ר 1 בודק בעיקר עמידה בהוראות:** קבצים, אחריות הפונקציות, ה-Endpoint, הפרמטרים ופורמט הפלט.
- **פיצ'ר 2 בודק בעיקר תכנון:** Flow, עיצוב DB, SRP, DRY ומבנה נקי.
- **אין ענישה כפולה.** כשל לוגי יורד מסעיף הפיצ'ר, ובעיה מבנית יורדת מ-Clean Code. מורידים בשניהם רק אם כל אחד חלש בפני עצמו.

וריאציות שמתקבלות בלי ניכוי

- שם משתנה הכתובת ב-`.env` של הלקוח (`SERVER_URL`, או `DOMAIN` ו-`PORT`), כל עוד הוא נטען דרך `config.py`.
- שמות פרמטרים שונים מההוראות, כל עוד הלקוח והשרת תואמים זה לזה.
- כל דרך לשליחת פרטי ההזדהות: headers, body או query params.
- `COUNT(*)` במקום `COUNT(DISTINCT animal_id)` – בגלל ההנחה שאותו טיפול לא נרשם פעמיים, שתיהן נותנות אותה תוצאה.
- מיקום סביר אחר לקובץ התכנון, אם הוא לא בדיוק `sql/design.txt`.

טבלת ציון

סעיף	מה נבדק	נקודות
פיצ'ר 1 – Client	בחירת סוג חיפוש, ולידציות, בקשת HTTP, פורמט הפלט, main כ-Flow בלבד	12
פיצ'ר 1 – Server + DB	Endpoint, חילוץ פרמטרים, JOIN, תשובה, 404, טעינת נתונים	15
פיצ'ר 2 – Authentication	רישום משתמש, hash לסיסמה, משתמש כפול, אימות לפני כל query	7
פיצ'ר 2 – Database Design	טבלאות, PK/FK, מניעת כפילות, טעינת ה-CSV, design.txt	10
פיצ'ר 2 – רישום טיפול	POST /treatments: בדיקת קיום, outcome חוקי, דחיית כפילות, INSERT	8
פיצ'ר 2 – Investigation	חיפוש פקח לפי id/name, ספירת חיות ייחודיות לפי שמורה, שם שמורה	16
Full Stack Flow	תואם מקצה לקצה Client → Server → DB → Server → Client	10
Clean Code / SRP / DRY	חלוקת אחריות, פונקציות קצרות, שמות ברורים, בלי שכפול	17
Git ועבודת צוות	מאגר משותף, branches, Pull Requests, commit-ים מכל חבר	5
סה"כ		**100**

פירוט Clean Code (17)

רכיב	נקודות
SRP ומבנה קוד	6
DRY	4
שמות וקריאות	3
פונקציות קצרות, main ו-Endpoints שמנהלים Flow בלבד	4

פירוט Git ועבודת צוות (5)

רכיב	נקודות
מאגר משותף שכל חברי הקבוצה מוגדרים בו	1
עבודה ב-branches לפי פיצ'ר או חבר צוות	1.5
מיזוג ל-main דרך Pull Requests	1.5
commit-ים מכל חבר צוות, עם הודעות ברורות	1

חלוקת סעיף ה-Flow (10)

עד 5 נקודות על Flow תקין בפיצ'ר 1, ועד 5 על Flow תקין בפיצ'ר 2 (רישום, רישום טיפול ותחקור). כך קבוצה שלא השלימה את פיצ'ר 2 לא מקבלת אפס על פיצ'ר 1 שעבד.

מדריך חומרה מהיר

סוג הבעיה	ניכוי טיפוסי	משמעות
תחביר או הקלדה	0.5–1	הכוונה ברורה, המימוש כמעט נכון
לוגיקה קטנה או מקרה קצה	1–2	כיוון נכון, אבל חלקי או שביר
דרישה חשובה חסרה	2–5	הפיצ'ר מזוהה, אבל חסר בו חלק נדרש
לוגיקת ליבה שגויה	4–8	הגישה לא עונה נכון על הפיצ'ר
בעיה מבנית גדולה	מסעיף Clean Code	פונקציות ארוכות, שכפול, ערבוב אחריות

פיצ'ר 1 – דוגמאות ניכוי

1. ולידציית קוד קולר חלקית | ניכוי מוצע: 0.5–1

```
def read_collar_code() -> str:
    while True:
        code = input("Collar code: ").strip()
        if code:
            return code
```

למה: יש לולאה ובדיקה שהערך לא ריק, אבל חסרה הבדיקה שהקוד מתחיל ב-COL-.

2. ID בלי לולאה ובלי בדיקת חיוביות | ניכוי מוצע: 1–2

```
def read_animal_id() -> int:
    return int(input("Animal ID: "))
```

למה: הסוג נכון, אבל הפונקציה לא חוזרת על הבקשה עד שמתקבל מספר שלם חיובי, וקלט לא מספרי יפיל את התוכנית.

3. main עושה הכול בעצמה | ניכוי מוצע: 2–3 (ומבנה – מ-Clean Code אם זה דפוס רחב)

```
def main():
    animal_id = input("ID: ")
    response = requests.get(SERVER_URL + "/animals/last-reserve",
                            params={"animal_id": animal_id})
    print(response.json())
```

למה: רעיון ה-HTTP נכון, אבל main מבצעת קלט, בקשה והדפסה בעצמה, ומדפיסה JSON גולמי במקום להשתמש בפונקציות המוכתבות.

4. חוזה בקשה שגוי | ניכוי מוצע: 3–5

```
requests.post(SERVER_URL + "/animal-reserve", json={"id": animal_id})
```

למה: ההוראות דורשות GET ל-animals/last-reserve/ עם query parameter. זו לא טעות תחביר אלא אי-עמידה בחוזה ה-API.

5. השרת מחזיר reserve_id במקום פרטי השמורה | ניכוי מוצע: 3–5

```
SELECT id, collar_code, species, last_seen_reserve_id
FROM animals
WHERE id = %s;
```

למה: החיה נמצאת, אבל אין JOIN לטבלת reserves, ולכן אי אפשר להחזיר שם שמורה, אזור ותיאור.

6. כיוון נכון, טעות קטנה בשאילתה | ניכוי מוצע: 1–2

```
SELECT a.id, a.collar_code, a.species, a.nickname,
       r.name, r.region, r.description
FROM animals a
JOIN reserves r ON a.last_seen_reserve_id = r.id
WHERE a.collar_code = ?;   -- placeholder of another driver
```

למה: השאילתה נכונה ביסודה. placeholder של דרייבר אחר, alias קטן או שם עמודה מעט שגוי – ניכוי קטן.

7. סינון על הטבלה הלא נכונה | ניכוי מוצע: 1.5–2

```
JOIN reserves r ON a.last_seen_reserve_id = r.id
WHERE r.id = %s
```

למה: בחיפוש לפי ID הסינון הוא על ה-id של השמורה במקום על ה-id של החיה. ה-JOIN נכון, אבל התוצאה שגויה.

פיצ'ר 2 – אימות ועיצוב DB

פיצ'ר 2 פתוח. מתגמלים תכנון סביר גם כשפרטי המימוש לא מושלמים. מורידים הרבה רק כשהעיצוב לא מאפשר לענות על השאלה או כשדרישת האבטחה לא מומשה ביסודה.

Authentication (7)

1. סיסמה נשמרת כטקסט גלוי | ניכוי מוצע: 3–4

```
with open("users.csv", "a") as file:
    file.write(f"{username},{password}\n")
```

למה: ה-hash הוא דרישת ליבה של האימות, לא עניין של סגנון.

2. hash קיים, טעות bytes/str באימות | ניכוי מוצע: 1

```
stored_hash = row[1]
if bcrypt.checkpw(password, stored_hash):
    return True
```

למה: הכיוון נכון, אבל חסר encode. מאחר שלא מריצים את הקוד, הניכוי קטן.

3. ה-query רץ לפני האימות | ניכוי מוצע: 2–3

```
result = count_animals_by_reserve(ranger_id)
if not authenticate(username, password):
    return {"error": "Unauthorized"}
return result
```

למה: הדרישה המפורשת היא לאמת לפני כל גישה ל-DB. שני החלקים קיימים, אבל הסדר שגוי.

4. משתמש כפול נרשם שוב | ניכוי מוצע: 1

```
def register_user(username, password):
    append_user(username, hash_password(password))
```

למה: הרישום וה-hash תקינים, אבל אין בדיקה ששם המשתמש לא קיים כבר.

Database Design (10)

5. כיוון נכון, FK אחד חסר | ניכוי מוצע: 1–2

```
rangers(id PK, name, specialty, years_of_service)
treatments(id PK, date, animal_id FK, ranger_id, reserve_id FK, outcome)
```

למה: המבנה כמעט מספיק, אבל ranger_id לא מוגדר כ-Foreign Key.

6. אין מניעת כפילות | ניכוי מוצע: 1–1.5

```
treatments(id PK, date, animal_id FK, ranger_id FK, reserve_id FK, outcome)
-- no UNIQUE (animal_id, ranger_id, reserve_id)
```

למה: המבנה עונה על השאלה, אבל לא מבטיח שאותו טיפול לא יירשם פעמיים, ו-design.txt לא מסביר איך זה נמנע. אם הבדיקה מתבצעת בקוד ומוסברת – אין ניכוי.

7. העיצוב לא מאפשר לענות על השאלה | ניכוי מוצע: 4–6

```
treatments(id PK, date, outcome)
```

למה: הטיפול לא מזהה חיה, פקח או שמורה, ולכן אי אפשר לחשב את התחקור.

8. design.txt חסר הסבר, המבנה תקין | ניכוי מוצע: 1

```
rangers(id PK, name, specialty, years_of_service)
treatments(id PK, date, animal_id FK, ranger_id FK, reserve_id FK, outcome,
           UNIQUE(animal_id, ranger_id, reserve_id))
```

למה: העיצוב נכון. ההסבר החסר הוא אי-עמידה בהוראות, לא סיבה להוריד מכל הסעיף.

פיצ'ר 2 – רישום טיפול ותחקור

רישום טיפול – POST /treatments (8)

1. INSERT בלי בדיקת קיום | ניכוי מוצע: 2–3

```
def add_treatment(data):
    cursor.execute("INSERT INTO treatments (...) VALUES (%s, %s, %s, %s, %s)", ...)
```

למה: ה-INSERT נכון, אבל אין בדיקה שהחיה, הפקח והשמורה קיימים, ואין טיפול בשגיאה. אם יש FK ב-DB והשגיאה נתפסת ומוחזרת – ניכוי קטן בלבד.

2. outcome לא נבדק | ניכוי מוצע: 1

```
outcome = data["outcome"]
insert_treatment(..., outcome)
```

למה: ערך שאינו healthy, treated או relocated יישמר. בדיקה בלקוח בלבד, או CHECK ב-DB, מספיקים בשביל ניקוד מלא.

3. ה-Endpoint לא מאובטח | ניכוי מוצע: 2–3

```
@app.post("/treatments")
def create_treatment(treatment: Treatment):
    return add_treatment(treatment)
```

למה: הכתיבה עצמה תקינה, אבל כל אחד יכול לרשום טיפול בלי הזדהות. הניכוי כאן, לא בסעיף האימות, כדי לא להעניש פעמיים.

Investigation (16)

4. לוגיקה נכונה | ניכוי מוצע: 0

```
SELECT r.name AS reserve_name, COUNT(DISTINCT t.animal_id) AS animals_count
FROM treatments t
JOIN reserves r ON r.id = t.reserve_id
WHERE t.ranger_id = %s
GROUP BY r.id, r.name;
```

למה: עונה ישירות על השאלה בחיפוש לפי id. בחיפוש לפי שם אפשר לאתר קודם את הפקח או להוסיף JOIN ל-rangers.

5. קיבוץ לפי פקח במקום לפי שמורה | ניכוי מוצע: 5–7

```
SELECT t.ranger_id, COUNT(DISTINCT t.animal_id)
FROM treatments t
GROUP BY t.ranger_id;
```

למה: זו תשובה לשאלה אחרת. הנדרש הוא מספר החיות בכל שמורה עבור פקח אחד.

6. reserve_id במקום שם השמורה | ניכוי מוצע: 1

```
SELECT t.reserve_id, COUNT(DISTINCT t.animal_id)
FROM treatments t
WHERE t.ranger_id = %s
GROUP BY t.reserve_id;
```

למה: הספירה נכונה, אבל הדרישה היא להחזיר את שם השמורה.

7. fetchone במקום fetchall | ניכוי מוצע: 1.5–2

```
cursor.execute(query, (ranger_id,))
row = cursor.fetchone()
```

למה: השאילתה נכונה, אבל מוחזרת רק השמורה הראשונה.

Full Stack Flow (10)

8. אי-התאמה בפרמטרים בין לקוח לשרת | ניכוי מוצע: 2–4

```
# Client
params={"ranger": value}
# Server
def investigate(ranger_id: int): ...
```

למה: כל צד סביר בפני עצמו, אבל הם לא מתחברים. הניכוי תלוי בכמה הפער בולט וקל לתיקון.

9. מפתח תשובה לא תואם | ניכוי מוצע: 1

```
# Server returns
{"reserve_name": "Gamla", "animals_count": 3}
# Client reads
row["reserve"]
```

למה: הארכיטקטורה והלוגיקה נכונות, ומפתח אחד לא עקבי. טעות אינטגרציה קטנה.

דוגמאות – Clean Code / SRP / DRY

כאן נמצאים הניכויים המבניים המשמעותיים. קוד יכול להיות נכון לוגית ועדיין לאבד כאן נקודות.

1. main עושה הכול | ניכוי מוצע: 4–5 מ-Clean Code

```
def main():
    username = input("Username: ")
    password = input("Password: ")
    # validate input, call HTTP endpoint,
    # parse JSON, print formatted output, handle errors
```

למה: main צריכה לנהל את ה-Flow, לא את הוולידציה, ה-HTTP, הפענוח וההצגה.

2. אותה ולידציה מועתקת | ניכוי מוצע: 2–3 מ-DRY

```
while not username.strip():
    username = input("Username: ")
while not ranger_name.strip():
    ranger_name = input("Ranger name: ")
```

למה: אותה בדיקת מחרוזת לא ריקה חוזרת. פונקציית עזר כמו read_non_empty(prompt) הייתה מונעת את השכפול. כך גם קריאת מזהים חיוביים חוזרת לחיה, לפקח ולשמורה.

3. Endpoint שמכיל אימות + SQL + עיצוב | ניכוי מוצע: 3–4 מ-SRP

```
@app.get("/rangers/treatments-by-reserve")
def investigate(...):
    # read users.csv, check bcrypt, connect to DB,
    # build SQL, execute, convert rows
    return result
```

למה: ה-Endpoint צריך בעיקר לתאם את הבקשה. האימות והשאילתה צריכים לשבת בשירותים נפרדים.

4. אימות משוכפל בשני ה-Endpoints | ניכוי מוצע: 1–2 מ-DRY

```
# in POST /treatments and in GET /rangers/... :
if not check_user(username, password):
    raise HTTPException(401)
```

למה: שורה אחת שחוזרת מקובלת. אם כל לוגיקת קריאת ה-CSV והבדיקה מועתקת לשני המקומות – זה שכפול.

5. שמות לא ברורים | ניכוי מוצע: 1–2

```
def f(x, y):
    a = db.execute(x, y)
    return a
```

למה: הקוד עשוי לעבוד, אבל השמות לא מסבירים את הכוונה. הניכוי על קריאות, לא על הלוגיקה.

6. מבנה נקי | ניכוי מוצע: 0

```
def main():
    search_type = choose_search_type()
    value = read_search_value(search_type)
    result = request_last_reserve(search_type, value)
    show_last_reserve(result)
```

למה: זה הסגנון הרצוי: פונקציות קצרות, שמות ברורים, אחריות אחת לכל פונקציה, ו-main שמנהלת רק את ה-Flow.

Git ועבודת צוות – דוגמאות

ניכוי מוצע	מה רואים במאגר
2.5–3	כל העבודה ב-commit אחד גדול, או רק על main
1.5	יש branches, אבל המיזוג ישירות ל-main בלי Pull Request
1	חבר צוות אחד בלי commit-ים כלל
0.5	הודעות commit כמו "update" או "fix" לאורך כל המאגר
לבקש קישור תקין לפני הבדיקה	המאגר לא נגיש לבודק או שהקישור שבור

תזכורת מהירה

הבעיה	פעולה
טעות הקלדה, כוונה ברורה	0.5–1
גישה נכונה, מקרה קצה שגוי	1–2
שלב נדרש חסר, הפיצ'ר מזוהה	2–4
לוגיקת ליבה עונה על שאלה אחרת	4–7
בעיית SRP או DRY גדולה	בעיקר מ-Clean Code
אותו פגם בדיוק	מורידים פעם אחת, במקום שבו הוא שייך

עיקרון מנחה: מתגמלים תכנון נכון וחשיבה ברורה. מורידים מעט על תחביר, והרבה יותר על לוגיקה שגויה ומבנה חלש.